

CSS Interview-Focused Notes

From CSS Fundamentals → Flexbox → CSS Grid

Structured for interviews, revision, and practical frontend development.

Coverage	Included
CSS Fundamentals	Selectors, cascade, specificity, inheritance, units, box model
Layout	Display, positioning, overflow, responsive design
Flexbox	Axes, alignment, wrapping, flex sizing, common traps
Grid	Tracks, lines, areas, fr, minmax(), auto-fit/auto-fill
Interview Prep	Common questions, traps, cheat sheet, must-memorize concepts

Core mental model: Normal Flow → Positioning → Flexbox (1D) → Grid (2D)

1. CSS Fundamentals

What is CSS?

CSS (Cascading Style Sheets) controls the presentation, styling, and layout of HTML elements.

```
p {  
  color: blue;  
  font-size: 18px;  
}
```

Ways to add CSS

- Inline: `<p style="color:red">Hello</p>`
- Internal: CSS inside a `<style>` element.
- External: `<link rel="stylesheet" href="style.css">` — preferred for maintainable projects.

CSS Syntax

```
selector {  
  property: value;  
}
```

Selector targets elements; property identifies what changes; value specifies the setting.

2. Selectors

Selector	Example	Purpose
Element	<code>p { }</code>	All matching elements
Class	<code>.card { }</code>	Elements with class card
ID	<code>#header { }</code>	Element with that ID
Universal	<code>* { }</code>	All elements
Grouping	<code>h1, h2, p { }</code>	Several selectors share rules
Attribute	<code>input[type="text"]</code>	Matches by attribute

Combinators

```
.container p /* descendant */  
.container > p /* direct child */  
h1 + p /* adjacent sibling */  
h1 ~ p /* general sibling */
```

Pseudo-classes

Represent a state or condition.

```
:hover  
:focus  
:active  
:visited  
:first-child  
:last-child  
:nth-child()  
:not()  
:checked  
:disabled
```

Pseudo-elements

Style a part of an element.

```
p::first-letter { ... }
.box::before {
  content: "★";
}
.box::after { ... }
```

Interview point: ::before and ::after generally need content to generate their pseudo-element content.

3. Cascade, Specificity & Inheritance

Cascade

When multiple declarations target the same property, CSS resolves conflicts through the cascade. Key factors include importance/origin, specificity, and source order.

```
p { color: blue; }
p { color: red; } /* wins: equal specificity, later rule */
```

Specificity

Relative priority	Typical example
Inline style	style="color:red"
ID	#title
Class / attribute / pseudo-class	.text, [type="text"], :hover
Element / pseudo-element	p, ::before

Note: This is a useful interview model, not the complete cascade algorithm. !important also changes normal priority and should not be used as a routine fix.

Inheritance

Some properties inherit from a parent to descendants.

- Common inherited properties: color, font-family, font-size, line-height.
- Properties such as margin, padding, border, width and height are generally not inherited.

4. Units & Values

Unit	Meaning / Use
px	Absolute CSS pixel unit
%	Relative percentage value
em	Relative to relevant element/context font sizing
rem	Relative to root element font size
vw / vh	Viewport width / viewport height

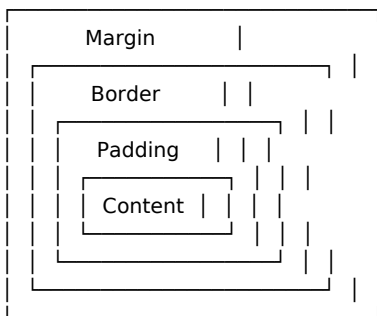
```
font-size: 2rem;
width: 50%;
width: 50vw;
height: 100vh;
```

Interview favorite: rem is often easier to reason about for scalable typography because it is based on the root font size.

5. Box Model

Every element is represented by the CSS box model:

Content → Padding → Border → Margin



box-sizing

```
/* default */
box-sizing: content-box;

/* easier sizing model */
*, *::before, *::after {
  box-sizing: border-box;
}
```

With content-box, declared width applies to content. With border-box, declared width/height includes padding and border.

Margin & Padding

- Margin: space outside the border.
- Padding: space inside the border.
- Four-value shorthand follows: top → right → bottom → left.
- Margin collapse: certain vertical margins in normal block layout can collapse instead of adding.

6. Display

Value	Key behavior
block	New line; dimensions can apply normally; typically fills available inline space
inline	Flows with text; width/height do not behave like block boxes
inline-block	Flows inline while accepting box dimensions
none	Removed from layout
flex	Flex formatting context
grid	Grid formatting context

display:none vs visibility:hidden: display:none removes the element from layout; visibility:hidden hides it while preserving its layout space.

7. Positioning

Value	Interview summary
static	Default normal-flow positioning
relative	Stays in normal flow; offsets visually move it; commonly establishes containing block for absolute descendants

Value	Interview summary
absolute	Removed from normal flow; positioned relative to its containing block
fixed	Typically positioned relative to the viewport; stays fixed during scrolling
sticky	Normal-flow behavior until a scroll threshold, then sticks within its scrolling/containing context

```
.parent {
  position: relative;
}
```

```
.child {
  position: absolute;
  top: 0;
  right: 0;
}
```

z-index

Controls stacking order where applicable. A large z-index does not automatically beat every element because stacking contexts matter.

8. Overflow, Text, Backgrounds & Borders

Overflow

```
overflow: visible;
overflow: hidden;
overflow: scroll;
overflow: auto;
```

Text

```
color: #333;
text-align: center;
text-decoration: underline;
text-transform: uppercase;
line-height: 1.5;
letter-spacing: 1px;
```

Background

```
background-color: black;
background-image: url("image.jpg");
background-position: center;
background-size: cover;
background-repeat: no-repeat;
```

cover: fills the container and may crop. contain: keeps the whole image visible and may leave empty space.

Borders

```
border: 1px solid black;
border-radius: 10px;
```

Opacity

opacity ranges from 0 (transparent) to 1 (fully opaque). It affects the entire element, including its contents.

9. CSS Variables & Useful Functions

```
:root {
  --primary-color: blue;
```

```
--spacing: 20px;
}

button {
  background: var(--primary-color);
  padding: var(--spacing);
  color: var(--text-color, black);
}
```

Sizing functions

```
width: calc(100% - 40px);
width: min(500px, 100%);
width: max(300px, 50%);
font-size: clamp(16px, 2vw, 32px);
```

clamp(min, preferred, max) is especially useful for fluid responsive sizing.

10. Responsive Design

```
@media (max-width: 768px) {
  .container {
    padding: 10px;
  }
}
```

Choose breakpoints based on where the layout needs to change, rather than blindly tying every breakpoint to a specific device name.

11. Flexbox

Flexbox is primarily a one-dimensional layout system. It arranges items along a row or column.

```
.container {  
  display: flex;  
}
```

Mental model: Flexbox → one dimension; Grid → two dimensions.

Main Axis & Cross Axis

The axes depend on flex-direction. With row, the main axis is horizontal; with column, the main axis is vertical.

```
flex-direction: row;  
flex-direction: column;
```

Core Flex Properties

Property	What it controls
flex-direction	Main-axis direction
justify-content	Distribution along main axis
align-items	Alignment along cross axis
align-content	Distribution of multiple flex lines
flex-wrap	Whether items can wrap
gap	Spacing between items
align-self	One item's cross-axis override
order	Visual ordering among flex items

justify-content

```
display: flex;  
justify-content: center;  
/* flex-start | center | flex-end |  
   space-between | space-around | space-evenly */
```

Interview rule: justify-content works along the main axis, not necessarily horizontally.

align-items

```
display: flex;  
align-items: center;  
/* stretch | flex-start | center |  
   flex-end | baseline */
```

Centering

```
.container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

Wrapping & Gap

```
.container {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 20px;  
}
```

flex-grow, flex-shrink, flex-basis ☐☐☐

Property	Meaning
flex-grow	Share of available positive free space
flex-shrink	How an item participates in shrinking when space is insufficient
flex-basis	Initial main-size contribution before free-space distribution
flex	Shorthand for grow, shrink, basis

```
.item {  
  flex: 1 1 0;  
}
```

Here: grow = 1, shrink = 1, basis = 0.

order & align-self

```
.item1 { order: 2; }  
.item2 { order: 1; }  
  
.special {  
  align-self: flex-end;  
}
```

Accessibility note: Visual order is not the same thing as DOM/source order. Be careful when using order.

Flexbox Interview Traps

- justify-content vs align-items: main axis vs cross axis.
- align-items vs align-content: items within a line/track vs distribution of multiple flex lines.
- flex-basis: relates to the main axis, so its effect depends on flex-direction.

12. CSS Grid ☐☐☐

CSS Grid is a two-dimensional layout system for rows and columns.

```
.container {  
  display: grid;  
  grid-template-columns: 200px 1fr 2fr;  
}
```

Grid Columns & Rows

```
grid-template-columns: 1fr 1fr;  
grid-template-rows: 100px 200px;
```

fr represents a fraction of available grid space.

repeat()

```
grid-template-columns: repeat(4, 1fr);
```

gap

```
gap: 20px;  
row-gap: 10px;  
column-gap: 20px;
```

Grid Template Areas ☐

```
.container {  
  display: grid;  
  grid-template-areas:  
    "header header"  
    "sidebar main"
```



```

    "footer footer";
}

.header { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main    { grid-area: main; }
.footer  { grid-area: footer; }

```

Grid Lines

If a grid has 3 columns, it has 4 column lines:

```

| col | col | col |
1    2    3    4

.item {
  grid-column: 1 / 3; /* spans 2 columns */
  grid-row: 1 / 3; /* spans 2 rows */
}

.other {
  grid-column: span 2;
}

```

minmax()

```

grid-template-columns:
  repeat(3, minmax(200px, 1fr));

```

Each track can be at least 200px and can grow up to 1fr.

Responsive Grid

```

.container {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(200px, 1fr));
  gap: 20px;
}

```

auto-fit vs auto-fill

Value	Conceptual behavior
auto-fit	Fits as many tracks as possible and can collapse empty tracks so existing items can expand
auto-fill	Attempts to preserve as many possible tracks as the available space allows, including empty tracks

Grid Alignment

Property	Purpose
justify-items	Aligns grid items within their grid areas along the inline axis
align-items	Aligns grid items within their grid areas along the block axis
justify-content	Distributes grid tracks within the container along the inline axis
align-content	Distributes grid tracks within the container along the block axis
place-items	Shorthand for align-items + justify-items

```

.container {
  place-items: center;
}

```

13. Flexbox vs Grid

Feature	Flexbox	Grid
Dimension	1D	2D
Main focus	Row or column	Rows + columns
Tracks	No grid tracks	Yes
Template areas	No	Yes
Alignment	Excellent	Excellent
Typical use	Component-level linear layouts	Page/section layouts and 2D layouts

They work together. For example: page layout → Grid; navbar → Flexbox; individual card internals → whichever model best matches the design.

14. Common Interview Questions

Q: What is the CSS box model?

Content → padding → border → margin.

Q: What is specificity?

A part of the cascade used to resolve competing declarations; selector specificity gives a relative priority.

Q: display:none vs visibility:hidden?

display:none removes the element from layout; visibility:hidden hides it while preserving layout space.

Q: relative vs absolute?

Relative stays in normal flow and can establish a containing block; absolute is removed from normal flow and positioned relative to its containing block.

Q: What is Flexbox?

A one-dimensional layout system for arranging items along a row or column.

Q: justify-content vs align-items?

justify-content distributes along the main axis; align-items aligns along the cross axis.

Q: What is Grid?

A two-dimensional layout system for rows and columns.

Q: What is 1fr?

One fractional share of available grid space.

Q: What does minmax() do?

Defines a minimum and maximum size for a grid track.

Q: auto-fit vs auto-fill?

Both generate repeated tracks; auto-fit can collapse empty tracks so existing items can expand, while auto-fill can preserve empty tracks.

15. CSS Interview Cheat Sheet

Selectors

.class
#id
element
*
A B
A > B
A + B
A ~ B
[attr="value"]

Box Model

content → padding → border → margin

box-sizing:
content-box
border-box

Position

static
relative
absolute
fixed
sticky
z-index

Display

block
inline
inline-block
none
flex
grid

Flexbox

display: flex
flex-direction
justify-content
align-items
align-content
flex-wrap
gap
flex-grow
flex-shrink
flex-basis
flex
order
align-self

Grid

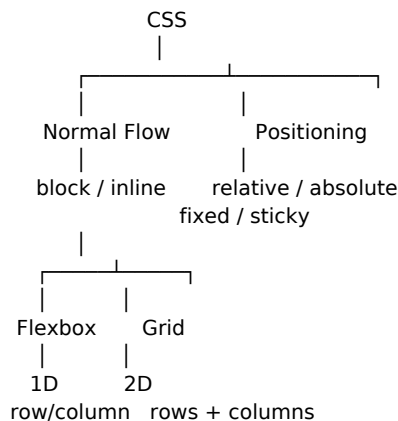
display: grid
grid-template-columns
grid-template-rows
grid-template-areas
grid-column
grid-row
grid-area
gap
fr
repeat()
minmax()
auto-fit
auto-fill
justify-items
align-items

justify-content
align-content
place-items

16. ☐ 15 Things to Memorize

- CSS box model
- box-sizing: border-box
- Specificity + cascade
- Inheritance
- Block vs inline vs inline-block
- display:none vs visibility:hidden
- Relative vs absolute positioning
- Fixed vs sticky
- z-index and stacking contexts
- Flexbox main vs cross axis
- justify-content vs align-items
- flex-grow, flex-shrink, flex-basis
- Flexbox vs Grid
- Grid tracks, lines, fr, repeat(), minmax()
- auto-fit vs auto-fill

17. One Mental Model



Final interview rule: Don't just memorize property names. Be able to explain which axis/context the property operates on, what happens to layout, and when you would use it.